Design Track

Interview Preparation Series • Professional Edition

System Design

05

Compiled with LaTeX • June 17, 2026

harsha.somisetty01@gmail.com

In This Edition

> **Highest-signal topic for senior loops.** Every FAANG design round tests the same core skills: decompose ambiguity, reason about scale, articulate trade-offs, defend decisions under pressure. This guide builds that capability from first principles.

1 Overview --- What it is

System design is the discipline of defining the **architecture, components, interfaces, and data flows** of a large-scale distributed software system to satisfy functional and non-functional requirements.

Unlike coding interviews (which have a single correct answer), design interviews are **open-ended**: the interviewer evaluates *how you think*, not just what you propose.

Key properties of a well-designed system:

- **Scalability** — handles growth in load/data without redesign
- **Reliability** — stays correct under failures
- **Availability** — serves requests despite partial failures
- **Maintainability** — easy to change and operate over time
- **Performance** — low latency, high throughput
- **Cost-efficiency** — doesn't over-engineer

First-principles mental model:

```
Requirements → Constraints → Trade-offs → Architecture
```

Every design decision is a trade-off. There is no ”best” architecture — only architectures that are best *for a given set of constraints*.

2 Why It Exists

The internet changed everything. In the 1970s–90s, software ran on a single machine. As usage grew:

1. **Single server** hits CPU/memory limits → need to scale
2. **Single database** becomes the bottleneck → need replication, sharding
3. **Single data center** risks outage → need geographic distribution
4. **Single deployment unit** makes change risky → need microservices
5. **Synchronous calls** create cascading failures → need async messaging

Each real-world pain point birthed a design pattern. Understanding *why* each pattern exists helps you apply it correctly and explain it convincingly.

3 Why FAANG Cares (be specific per company)

Compar	What they're really testing	Key signals they look for
Google	Can you design at planetary scale? (Maps, Search, YouTube)	Bigtable/Spanner-style thinking, eventual consistency comfort, data locality, Borg-like resource scheduling

Compar	What they're really testing	Key signals they look for
Meta	Social graph scale (3B+ users), read-heavy feeds, real-time notifications	EdgeRank-style ranking, TAO graph DB, async fanout, infrastructure cost awareness
Amazon	Service-oriented culture, failure isolation, "two-pizza team" ownership	Six-pager thinking, blast radius minimization, DynamoDB/SQS/SNS, operational excellence
Apple	Privacy, device–cloud sync, iCloud-scale storage	End-to-end encryption architecture, conflict resolution (CRDT), offline-first design
Netflix	Chaos engineering mindset, streaming at scale, global CDN	Hystrix/resilience patterns, Open Connect CDN, A/B experimentation infrastructure
Microsc	Azure integration, enterprise reliability SLAs, Azure-native patterns	Multi-tenant isolation, RBAC, hybrid cloud, Teams-scale real-time messaging

Universal FAANG signal: Can you identify bottlenecks *before* being asked? Can you quantify trade-offs numerically? Do you think about failure modes proactively?

4 The System Design Interview Framework

> **Use this exact playbook in every design round. Stick to the sequence.**

4.1 Step 1: Clarify Requirements (3--5 min)

Never assume. Ask:

Functional requirements:

- What are the core features? (e.g., "Can users like posts? Comment? Follow?")
- What is out of scope? ("Do we need DMs? Stories?")
- Who are the users? (consumers, businesses, developers?)

Non-functional requirements:

- Scale: DAU, QPS, storage growth per year
- Latency: p99 read latency? write latency?
- Consistency: is eventual consistency acceptable? (news feed: yes; bank transfer: no)
- Availability: 99.9%? 99.99%? (8.7 hrs vs 52 min downtime/year)
- Durability: can we lose data? (never for payments)

Constraints:

- Budget, team size, timeline (rarely asked but good to mention you're aware)

Candidate phrase: *"Before I dive in, let me make sure I understand the requirements. I'll ask a few questions — stop me if I'm going too deep."*

4.2 Step 2: Capacity Estimation (3--5 min)

Back-of-envelope. Show your work. Round aggressively.

```
Example: Twitter
- DAU: 300M users
- Each reads 100 tweets/day → 300M × 100 / 86400 ≈ 350K reads/sec
- Each posts 1 tweet/10 days → 300M × 0.1 / 86400 ≈ 350 writes/sec
- Tweet size: 140 chars + metadata ≈ 300 bytes
- Storage: 350 writes/sec × 300 bytes × 86400 × 365 ≈ 3.3 TB/year
- Media (10% have images, 200KB avg): 350 × 0.1 × 200KB × 86400 × 365 ≈ 220 TB/year
```

Always estimate: QPS (read + write), storage/year, bandwidth, memory for cache.

4.3 Step 3: Define the API (2--3 min)

Sketch the core REST/RPC endpoints:

```
POST   /tweets            → create tweet
GET    /timeline/{userId} → get home timeline
POST   /follow            → follow user
GET    /user/{userId}     → get user profile
```

This forces you to crystallize exactly what the system does and surfaces data model decisions early.

4.4 Step 4: Data Model (3--5 min)

Choose SQL vs NoSQL. Define key entities and relationships.

```
User:    user_id (PK), username, email, created_at
Tweet:   tweet_id (PK), user_id (FK), content, media_url, created_at
Follow:  follower_id, followee_id, created_at (composite PK)
Feed:    user_id, tweet_id, score, created_at  ← for pre-computed feeds
```

Explain *why* each field exists and what queries drive the schema.

4.5 Step 5: High-Level Architecture (5--8 min)

Draw the major components and data flows. Always include:

```
Client → CDN → API Gateway → Load Balancer → App Servers → Cache → DB
                                                    ↓
                                            Message Queue → Workers
```

Name each component and explain its role. Don't over-engineer at this stage.

4.6 Step 6: Deep Dive (10--15 min)

Pick 2–3 of the hardest sub-problems and go deep. Let the interviewer guide you, but have opinions:

- "The hardest part of this system is X. Let me explain why and how I'd solve it."
- Common deep-dives: fan-out on write vs read, cache invalidation, sharding strategy, consistency guarantees.

4.7 Step 7: Identify Bottlenecks & Scale (3--5 min)

Proactively find your own system's weaknesses:

- **Single points of failure:** Where does the system go down if one component fails?
- **Hot spots:** Which users/keys get disproportionate traffic? (Celebrity problem)
- **Write amplification:** Fan-out writes cost more than fan-out reads at some scales.
- **Consistency vs latency:** Where are you making that trade-off explicitly?

Candidate phrase: *"Now let me stress-test my own design. Here are the three places I'd worry about at scale..."*

5 Core Concepts

5.1 Scalability

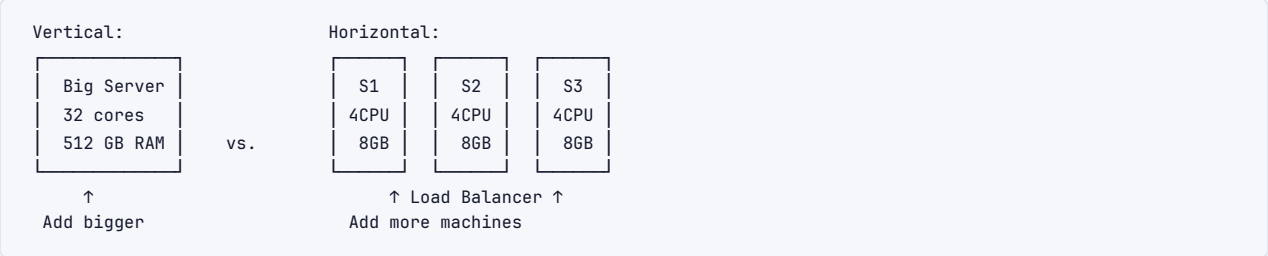
Definition: Ability to handle increased load by adding resources.

Vertical scaling (scale up): Add more CPU/RAM/disk to existing machine.

- Pros: Simple, no code changes, strong consistency
- Cons: Hardware limits, expensive, single point of failure, requires downtime

Horizontal scaling (scale out): Add more machines.

- Pros: Theoretically unlimited, commodity hardware, fault tolerant
- Cons: Requires stateless application tier, data distribution complexity



Interview takeaway: Start with vertical, switch to horizontal when you hit limits or need HA. Nearly all large-scale systems are horizontally scaled.

Stateless vs Stateful:

	Stateless	Stateful
Definition	Each request contains all needed context	Server stores session state between requests
Scaling	Trivially horizontal (any server handles any request)	Hard (sticky sessions, session replication)
Failure recovery	Easy (just route elsewhere)	Hard (state is lost)
Examples	REST APIs, CDN edge nodes	WebSocket connections, game servers, DB connections

Key insight: Make your application tier **stateless** by moving state to an external store (cache, DB). This is what makes horizontal scaling work.

5.2 Load Balancing

Definition: Distribute incoming traffic across multiple servers to avoid overloading any single one.

Algorithms:

Algorithm	How it works	Best for
Round Robin	Each server gets requests in rotation	Uniform server capacity, stateless requests
Weighted Round Robin	Servers get proportional share based on capacity	Mixed server hardware
Least Connections	Route to server with fewest active connections	Long-lived connections (WebSockets)
IP Hash	Hash client IP → sticky routing	Session persistence without external store
Random	Pick random server	Simple, uniform load

Algorithm	How it works	Best for
Least Response Time	Route to fastest responding server	Latency-sensitive workloads

Layer 4 vs Layer 7:

	L4 (Transport)	L7 (Application)
Operates at	TCP/UDP level	HTTP/HTTPS level
Sees	IP, port, protocol	URLs, headers, cookies, body
Speed	Faster (less processing)	Slower (more processing)
Flexibility	Less (can't inspect content)	More (route by URL, A/B test)
Examples	AWS NLB, HAProxy TCP	AWS ALB, Nginx, Envoy

Health checks: LBs continuously probe backend servers. Remove unhealthy instances automatically.

Interview takeaway: Use L7 LB for HTTP microservices (route by path/header). L4 for raw TCP (databases, game servers). Always have ≥2 LB instances for HA.

5.3 Caching

Definition: Store frequently accessed data in fast storage (memory) to reduce latency and backend load.

Why cache? Memory access: ~100ns. Disk: ~10ms. Network DB query: ~1-10ms. Cache hit: ~0.5ms local, ~1ms Redis.

Caching Strategies

1. Cache-Aside (Lazy Loading)

```
Read:                          Write:
App checks cache               App writes to DB
  ↓ miss                       App invalidates cache
App reads from DB              (next read repopulates)
App writes to cache
App returns data
```

- Pros: Only cache what's needed, cache failures don't break reads
- Cons: Cache miss is expensive (3 trips), stale data possible, thundering herd on cold start
- **Use when:** Read-heavy, data not always needed

2. Write-Through

```
Write:                         Read:
App writes to cache            Check cache → hit → return
Cache synchronously writes to DB  (always fresh)
Return success
```

- Pros: Cache always consistent with DB, no stale reads
- Cons: Write latency higher (2 writes), cache may hold rarely-read data
- **Use when:** Write + read same data frequently, consistency critical

3. Write-Back (Write-Behind)

```
Write:                          Background:
App writes to cache             Cache async flushes to DB
Return success immediately      (batched writes)
```

- Pros: Lowest write latency, batch DB writes (fewer I/O ops)
- Cons: Risk of data loss if cache fails before flush, complex recovery
- **Use when:** High write throughput, can tolerate small data loss window (analytics, logging)

4. Write-Around

```
Write: App writes directly to DB (bypasses cache)
Read:  Cache miss → read from DB → populate cache
```

- Pros: Cache not polluted with write-once data
- Cons: First read always misses
- **Use when:** Data written once, rarely read back soon (file uploads, logs)

Cache Eviction Policies

Policy	Rule	Best for
LRU (Least Recently Used)	Evict item not accessed longest	General purpose, temporal locality
LFU (Least Frequently Used)	Evict item accessed least times	Stable "hot" data (popular items stay)
MRU (Most Recently Used)	Evict most recently used item	Scans (last item won't be needed again)
FIFO	Evict oldest inserted item	Simple, predictable
Random	Evict random item	Simple approximation of LRU
TTL	Evict after time-to-live expires	Time-sensitive data

LRU Implementation: Doubly-linked list + HashMap. O(1) get and put. Classic interview question.

Cache Invalidation

The hardest problem in caching. Three strategies:

1. **TTL-based:** Set expiry time. Simple but stale until TTL expires.
2. **Event-driven:** When DB changes, publish invalidation event to cache. Complex but fresh.
3. **Write-through invalidation:** On write, update cache atomically. Consistent but slower writes.

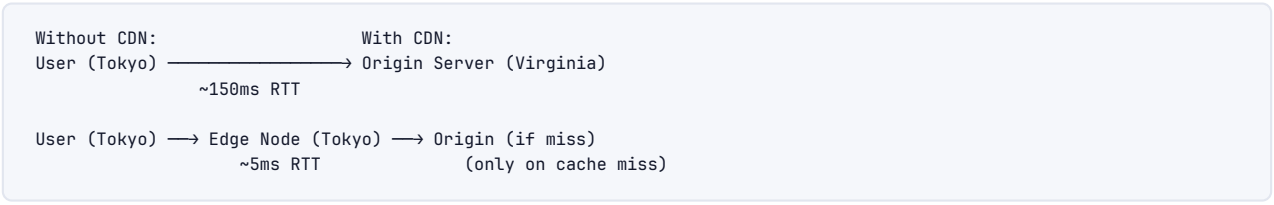
Cache stampede / Thundering Herd: When a popular cached item expires, thousands of requests simultaneously hit the DB. Solutions:

- **Probabilistic early expiration:** Refresh cache slightly before TTL with some probability
- **Mutex/lock:** First request acquires lock, others wait, lock-holder populates cache
- **Background refresh:** Async refresh before TTL, serve stale during refresh

Interview takeaway: Cache-aside is the most common pattern. Always discuss TTL strategy and cache invalidation — interviewers love this. Know LRU vs LFU trade-offs.

5.4 CDNs (Content Delivery Networks)

Definition: Geographically distributed network of servers (edge nodes/PoPs) that cache and serve content close to end users.



What CDNs serve: Static assets (images, CSS, JS, videos), but also dynamic content (API responses at edge), WebSocket connections at edge.

Push vs Pull CDN:

	Push CDN	Pull CDN
How content gets to edge	You upload/push to CDN	CDN fetches from origin on first request
Control	Full control, pre-populate	Automatic, reactive
Storage cost	Higher (must store all content at all edges)	Lower (only popular content cached)
Staleness	Manual invalidation needed	TTL-based, auto-expire
Best for	Large files known in advance (videos, software)	Unpredictable access patterns, websites

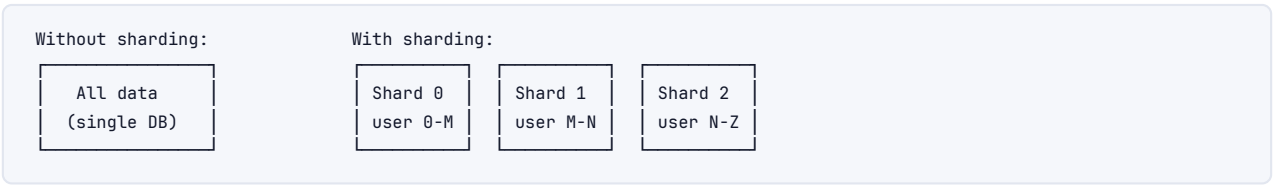
Key CDN features:

- **TLS termination at edge** → reduces SSL handshake cost for users
- **DDoS protection** → absorb attacks at edge before reaching origin
- **Edge compute** → run code at edge nodes (Cloudflare Workers, Lambda@Edge)
- **Geo-blocking** → restrict by country at network level

Interview takeaway: CDN = first line of defense for scale. Serve all static assets via CDN. For dynamic content, push DB-query results to CDN with short TTLs. Always mention CDN early in a design.

5.5 Sharding (Horizontal Partitioning)

Definition: Split a large dataset across multiple database nodes, each holding a subset of the data.



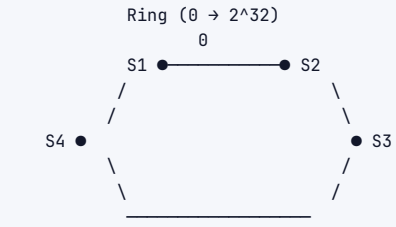
Sharding strategies:

Strategy	How	Pros	Cons
Range-based	Shard by key range (A-F, G-M, N-Z)	Simple, range queries easy	Hot spots if data not uniformly distributed
Hash-based	shard = hash(key) % N	Uniform distribution	Range queries hit all shards, resharding is painful
Directory-based	Lookup table maps key → shard	Flexible, easy to move data	Lookup table becomes bottleneck/SPOF
Geographic	Shard by user location	Latency, compliance (GDPR)	Cross-geo queries expensive

Consistent Hashing: Solves the resharding problem of hash-based sharding.

```
Normal hashing: shard = hash(key) % N
  → If N changes: almost ALL keys reassigned (massive rebalancing)

Consistent hashing:
  → Keys and servers mapped to a ring (0 to 2^32)
  → Each key assigned to the nearest server clockwise
  → When a server is added/removed: only (K/N) keys move
```



Virtual nodes (vnodes): Assign each physical server multiple positions on the ring → better load balancing, smoother failover.

Sharding pitfalls:

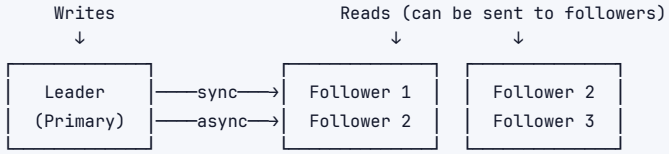
- **Cross-shard joins:** Joins across shards are expensive → denormalize or avoid
- **Cross-shard transactions:** Distributed transactions (2PC) are slow and complex
- **Hot shards:** Celebrity users, trending content → route to overloaded shard
- **Resharding:** Painful with hash-based, plan for it with consistent hashing

Interview takeaway: Sharding is the last resort after read replicas, caching, and indexing. Use consistent hashing to avoid resharding pain. Know the hot-spot problem and mitigation (add shard for celebrity, use virtual nodes).

5.6 Replication

Definition: Maintain copies of the same data on multiple machines for redundancy and read scaling.

Leader-Follower (Primary-Replica)



- All writes go to leader
- Followers replicate asynchronously (usually) or synchronously
- Reads can be distributed across followers → read scaling
- Automatic failover: a follower promoted to leader on primary failure

Sync vs Async replication:

	Synchronous	Asynchronous
Write latency	Higher (wait for replica ACK)	Lower (return after leader write)
Data loss risk	Zero (if replica fails, write fails)	Non-zero (unflushed writes lost)

	Synchronous	Asynchronous
Availability	Lower (writes blocked if replica down)	Higher
Typical use	Financial transactions, critical data	Social feeds, analytics

Leader-Leader (Multi-Primary / Active-Active)

- Both leaders accept writes
- Conflict resolution needed (last-write-wins, vector clocks, CRDTs)
- Best for geo-distributed systems where each region serves local writes
- Conflict resolution is the hard part

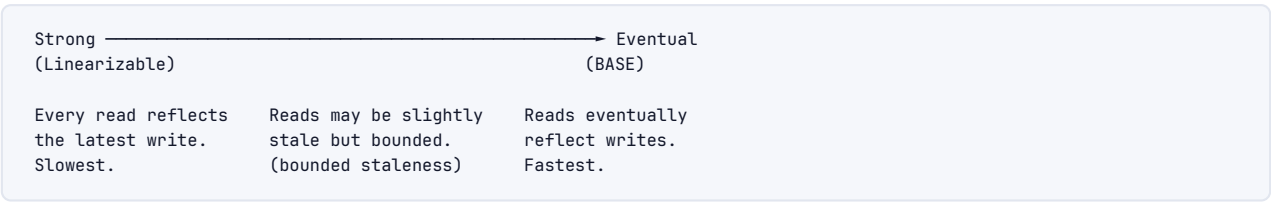
Replication lag: Followers may lag behind leader. Read-your-own-writes problem: write to leader, immediately read from follower → stale data.

Solutions to replication lag:

- Read own writes from leader
- Track replication position, route reads to followers that are caught up
- Accept eventual consistency

5.7 Consistency Models

Definition: Rules that define what values are valid to return for reads after writes.



Model	Guarantee	Example
Linearizability (Strong)	All ops appear to execute atomically at a single point in time	Single-node DB, Zookeeper
Sequential Consistency	All ops appear in some sequential order consistent per-process	Older CPUs, some distributed DBs
Causal Consistency	Causally related ops seen in order; concurrent ops may differ	Amazon Dynamo-style
Eventual Consistency	Given no new writes, all replicas converge eventually	DNS, Cassandra, DynamoDB default
Read-your-writes	You always see your own writes	Session-level guarantee
Monotonic reads	Once you read a value, you never read an older one	Prevents time-traveling reads

ACID vs BASE:

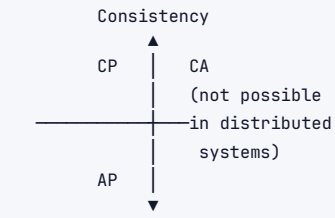
	ACID (SQL)	BASE (NoSQL)
Stands for	Atomicity, Consistency, Isolation, Durability	Basically Available, Soft-state, Eventually consistent
Consistency	Strong (serializable)	Eventual
Availability	Sacrificed during partition	Maintained
Performance	Lower throughput	Higher throughput
Use case	Banking, inventory, orders	Social feeds, caches, analytics

5.8 CAP Theorem

Statement: A distributed system can guarantee at most **two** of these three properties simultaneously:

- **C**onsistency — Every read sees the most recent write (all nodes agree on the same data)
- **A**vailability — Every request gets a non-error response (system is always up)
- **P**artition tolerance — System continues to operate despite network partition (lost messages)

Key insight: Network partitions *will* happen in any real distributed system. Therefore, you *must* tolerate partitions. The real choice is **CP vs AP**:



Choice	Behavior during partition	Examples
CP (Consistency + Partition)	Return error rather than stale data	HBase, Zookeeper, MongoDB (default config), Redis Cluster
AP (Availability + Partition)	Return stale data rather than error	Cassandra, DynamoDB, CouchDB, DNS

Real-world nuance: CAP is a spectrum, not binary. Modern systems use tunable consistency (Cassandra's `QUORUM`, `ONE`, `ALL` read/write levels).

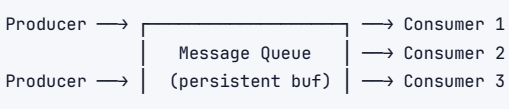
PACELC Extension: Even when no partition:

- **PAC:** During Partition → choose Availability or Consistency
- **ELC:** Else → Latency vs Consistency trade-off

Interview takeaway: "P is always required. The real choice is CP vs AP. Pick based on business need: banking = CP, social feed = AP. Always mention PACELC for senior roles."

5.9 Message Queues

Definition: Middleware that decouples producers (senders) and consumers (receivers) via an async buffer.



Why use message queues?

1. **Decoupling:** Producer doesn't need to know about consumers
2. **Load leveling:** Queue absorbs burst traffic, consumers process at their pace
3. **Resilience:** If consumer is down, messages wait in queue (not lost)
4. **Async processing:** Return response to user immediately; process in background
5. **Fan-out:** One message delivered to multiple consumers

Core concepts:

- **Producer:** Publishes messages
- **Queue/Topic:** Buffer that holds messages
- **Consumer/Subscriber:** Reads and processes messages
- **ACK:** Consumer acknowledges message after processing; queue deletes it
- **Dead Letter Queue (DLQ):** Messages that fail repeatedly go here for inspection
- **At-least-once delivery:** Message may be delivered multiple times → consumers must be **idempotent**
- **At-most-once:** Message delivered 0 or 1 times → no retries, possible loss
- **Exactly-once:** Delivered exactly once → hardest, expensive, often approximated

Kafka vs RabbitMQ vs SQS

Feature	Apache Kafka	RabbitMQ	AWS SQS
Type	Distributed log (pub/sub)	Message broker (queue)	Managed queue
Ordering	Partition-level ordering	FIFO queue variant	FIFO queue variant
Retention	Configurable (days/weeks)	Until consumed	Up to 14 days
Replay	Yes (seek to offset)	No	No
Throughput	Very high (millions/sec)	High (tens of thousands/sec)	High (managed)
Consumer model	Pull (consumer controls offset)	Push (broker pushes)	Pull
Routing	Topics + partitions	Flexible routing (exchanges)	Simple queue
Durability	Persistent (disk)	Persistent or transient	Persistent
Best for	Event streaming, log aggregation, high throughput	Complex routing, RPC patterns, task queues	Serverless, AWS-native, simple queues
Operations	Complex (Zookeeper, brokers)	Moderate	None (managed)

Kafka deep dive:

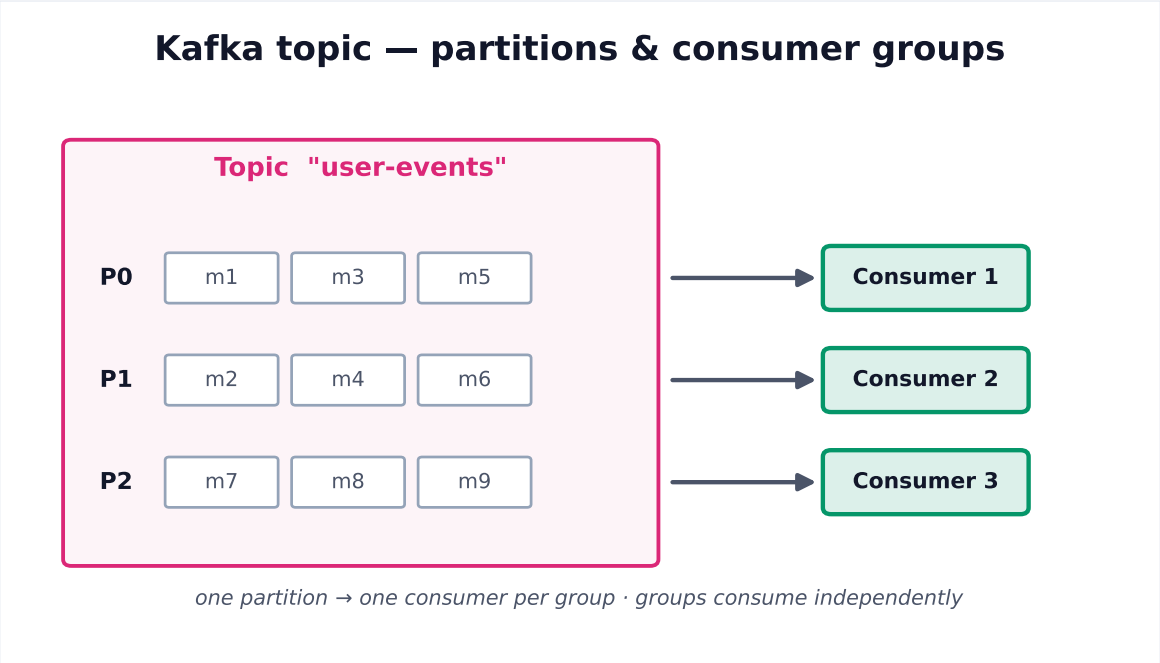
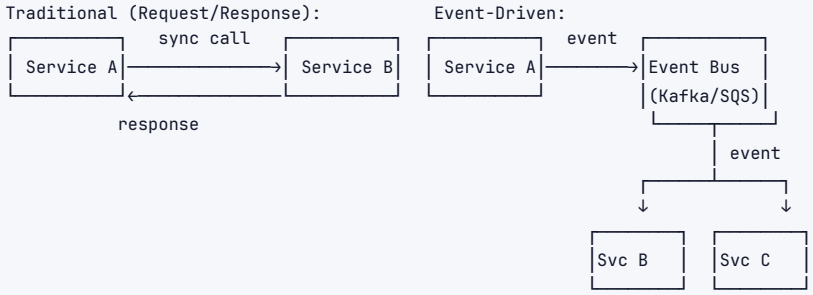


Figure 1. A topic is split into partitions for parallelism. Within a consumer group each partition is read by exactly one consumer; different groups read the same data independently.

Kafka use cases: Event sourcing, change data capture (CDC), log aggregation, stream processing, activity tracking.

5.10 Event-Driven Architecture

Definition: System where components communicate by emitting and reacting to events rather than direct calls.

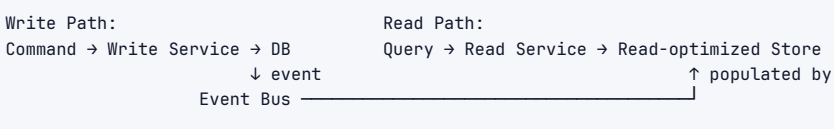


Patterns:

Event Sourcing: Store state changes as a sequence of events (not the current state).

- Audit log built in, can replay events to rebuild state
- Example: Bank account balance = sum of all transactions
- Cons: Querying current state requires replay, complex

CQRS (Command Query Responsibility Segregation): Separate read and write models.



- Write DB optimized for writes (normalized)
- Read DB optimized for reads (denormalized, cached)

- Eventual consistency between write and read sides

Saga Pattern: Manage distributed transactions across microservices via events.

```
Choreography Saga:
OrderService —[OrderCreated]→ PaymentService —[PaymentCompleted]→ ShippingService
                              —[PaymentFailed]→ OrderService (compensate)

Orchestration Saga:
Orchestrator → PaymentService → Orchestrator → ShippingService → ...
```

5.11 Microservices

Definition: Architectural style where an application is built as a suite of small, independently deployable services, each owning its data and communicating via APIs.

Monolith vs Microservices

	Monolith	Microservices
Deployment	Single unit	Independent per service
Scaling	Scale whole app	Scale individual services
Tech stack	Uniform	Polyglot (each service chooses)
Development	Simpler (one codebase)	Complex (distributed system)
Testing	Easier (unit + integration)	Hard (contract tests, integration)
Failure isolation	Poor (one bug can crash all)	Good (blast radius contained)
Data management	Shared DB (simpler)	DB per service (complex)
Latency	No network hops	Network calls between services
Team org	Coupled, coordination needed	Autonomous teams (Conway's Law)
When to use	Early stage, small team	Scale, large teams, clear domain boundaries

Conway's Law: *"Organizations design systems that mirror their communication structure."* Microservices enable team autonomy.

Microservices anti-patterns:

- **Nano-services:** Too fine-grained → too many network calls
- **Distributed monolith:** Services tightly coupled, deployed together → worst of both worlds
- **Shared database:** Multiple services share same DB → coupling through data

Service Communication

Synchronous:

- REST (HTTP/JSON) — simple, universal, stateless
- gRPC (HTTP/2 + Protobuf) — fast, typed, streaming, better for internal services
- GraphQL — flexible queries, avoid over/under-fetching

Asynchronous:

- Message queue (Kafka, SQS) — decoupled, resilient
- Event bus — fan-out, event-driven
- Webhooks — callback when event occurs

Sync vs Async:

	Synchronous	Asynchronous
Coupling	Tight (caller waits)	Loose (fire and forget)
Latency	Direct (low if service is fast)	Added queue latency
Availability	Caller fails if callee fails	Resilient (queue absorbs failure)
Complexity	Simple	Complex (eventual consistency, DLQ)
Use case	Query for data, need immediate response	Long-running tasks, fan-out, decoupling

Service Mesh (Istio, Linkerd): Infrastructure layer handling service-to-service communication: mTLS, observability, load balancing, circuit breaking — without changing application code.

Circuit Breaker Pattern: Prevent cascading failures.

```
CLOSED (normal) → fails exceed threshold → OPEN (reject requests) →
                                          timeout expires →
HALF-OPEN (probe) → success → CLOSED  /  failure → OPEN
```

5.12 Rate Limiting

Definition: Control the rate of requests to prevent abuse, ensure fair use, and protect system resources.

Algorithms:

Algorithm	How	Pros	Cons
Token Bucket	Tokens added at rate r, bucket capacity c. Request consumes token.	Allows bursts up to c. Smooth average.	Complex state
Leaky Bucket	Requests enter bucket, drain at fixed rate. Overflow dropped.	Smooth, constant output rate	Strict, no burst allowed
Fixed Window Counter	Count requests in fixed time window (e.g., per minute).	Simple	Edge problem: 2x rate possible at window boundary
Sliding Window Log	Log timestamps of all requests, count in last N seconds.	Accurate, no edge problem	Memory: O(requests)
Sliding Window Counter	Approximate using current + previous window counts, weighted.	Memory efficient, accurate	Approximation

Where to implement:

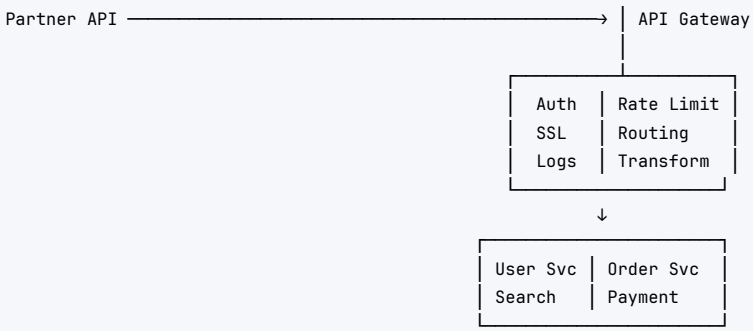
- Client-side (throttle outgoing requests)
- API Gateway level (before hitting any service)
- Per-service (business logic limits)
- Per-user, per-IP, per-API-key, per-endpoint

Redis implementation: `INCR key` + `EXPIRE key 60` → atomic counter per user per minute.

5.13 API Gateway

Definition: Single entry point for all client requests. Handles cross-cutting concerns.

```
Mobile ──────────────────────────────────────────────┐
Web ─────────────────────────────────────────────────→ │
```



API Gateway responsibilities:

- Authentication/Authorization (JWT validation, OAuth)
- Rate limiting and throttling
- SSL/TLS termination
- Request routing (to correct microservice)
- Load balancing
- Request/response transformation
- Logging, metrics, tracing
- Caching (edge caching)
- API versioning

Examples: AWS API Gateway, Kong, Nginx, Envoy, Traefik

5.14 Database Choice: SQL vs NoSQL

	SQL (Relational)	NoSQL
Data model	Tables, rows, columns	Document, key-value, wide-column, graph
Schema	Fixed, defined upfront	Flexible, schema-less
Transactions	ACID, multi-row	BASE, limited (improving)
Joins	Yes, powerful	No (denormalize or application-level)
Scaling	Vertical (primarily), read replicas	Horizontal (built-in)
Query language	SQL (standardized)	Proprietary APIs
Consistency	Strong	Eventual (usually)
Use cases	E-commerce, banking, ERP	Social feeds, real-time analytics, IoT
Examples	PostgreSQL, MySQL, Oracle	Cassandra, MongoDB, DynamoDB, Redis

NoSQL subtypes:

Type	Data model	Best for	Examples
Key-Value	key → value	Caching, sessions, leaderboards	Redis, DynamoDB, Memcached
Documen	key → JSON doc	User profiles, catalogs, CMS	MongoDB, CouchDB, Firestore
Wide-Column	row key → dynamic columns	Time series, IoT, event logs	Cassandra, HBase, BigTable

Type	Data model	Best for	Examples
Graph	Nodes + edges	Social networks, recommendations, fraud detection	Neo4j, Amazon Neptune
Time Series	Timestamp + metrics	Monitoring, metrics, IoT	InfluxDB, TimescaleDB, Prometheus

Interview takeaway: Start with SQL. Use NoSQL when you need: horizontal scale, flexible schema, specific access patterns SQL handles poorly (graph traversal, time series). Don't use NoSQL "because it's faster" — profile first.

5.15 Idempotency

Definition: An operation is idempotent if applying it multiple times has the same effect as applying it once.

```
Idempotent:      PUT /users/123  { name: "Alice" }  → same result each time
Not idempotent:  POST /orders    (creates new order each time)
Not idempotent:  counter++       (increments each call)
```

Why it matters: Networks are unreliable. Requests may be retried. If your operation isn't idempotent, retries cause duplicate side effects (double charges, duplicate records).

Implementation patterns:

- **Idempotency key:** Client sends unique key with request. Server stores key → result mapping. On retry, return stored result.
- **Natural idempotency:** Use PUT instead of POST for upserts. Use idempotent math (set value, not increment).
- **Deduplication window:** Store recently processed message IDs in Redis with TTL. Reject duplicates.

HTTP method idempotency:

Method	Idempotent	Safe (no side effects)
GET	Yes	Yes
HEAD	Yes	Yes
PUT	Yes	No
DELETE	Yes	No
POST	No	No
PATCH	No	No

6 Architecture / Diagrams

6.1 Typical Scalable Web Architecture

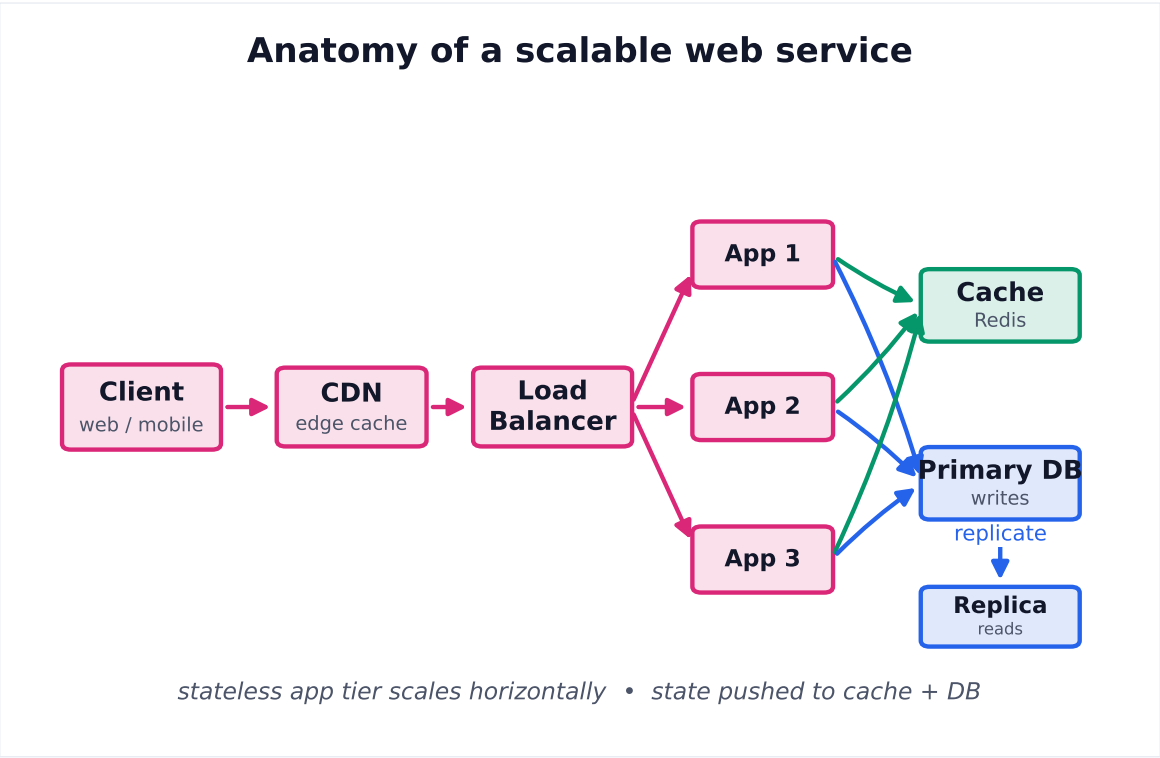
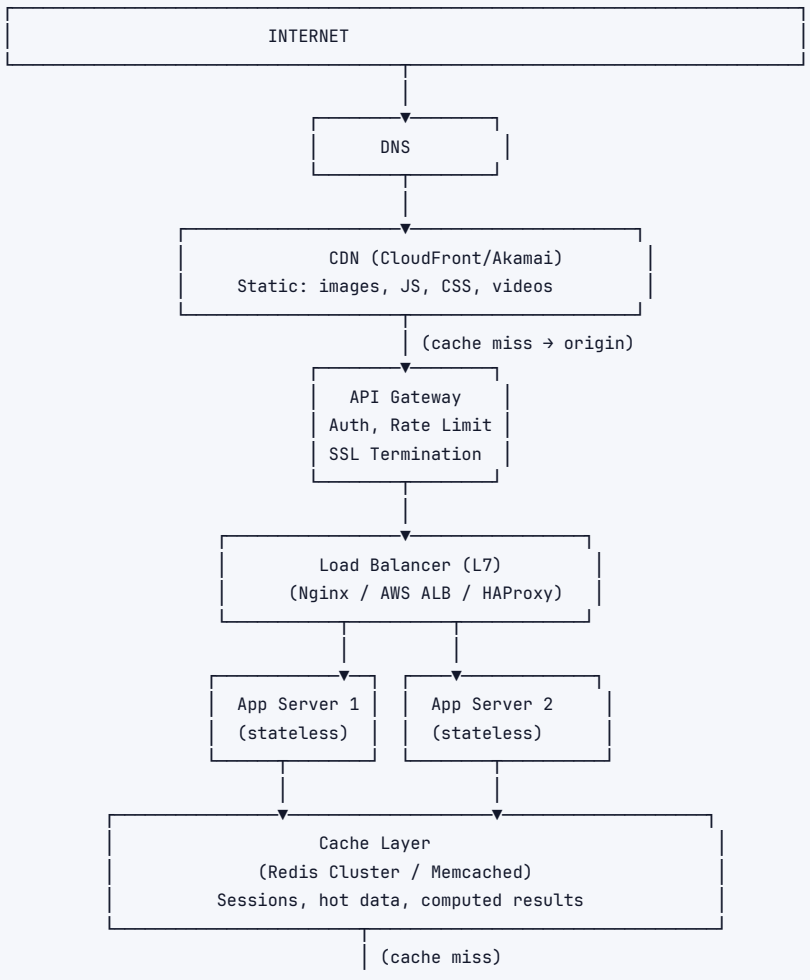
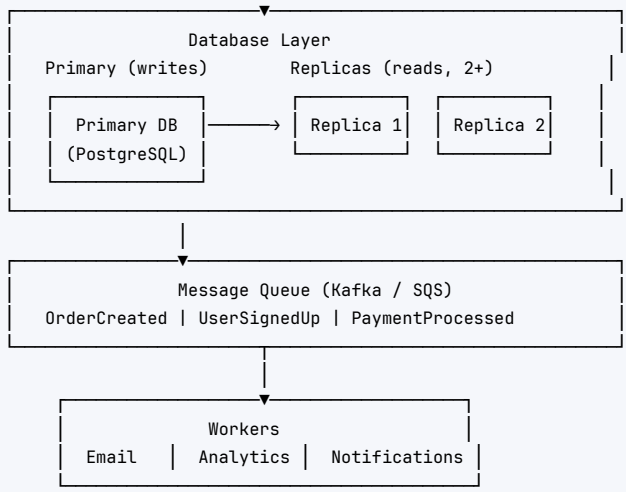
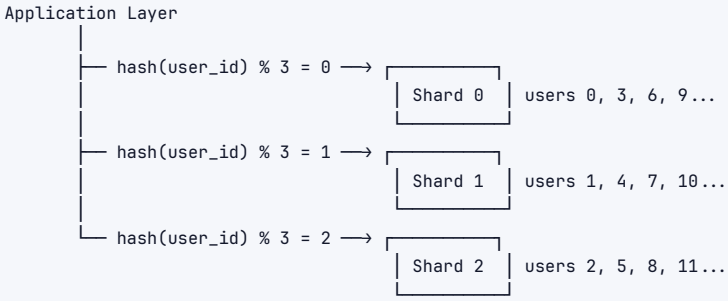


Figure 2. The canonical pattern: stateless app servers behind a load balancer, a CDN for static edge content, a cache to absorb reads, and a primary DB with read replicas.

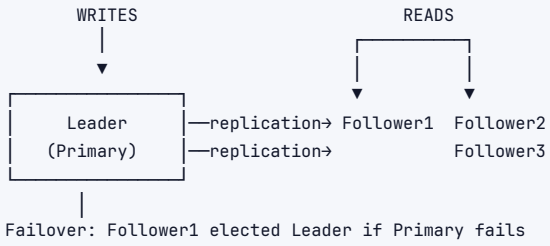




6.2 Sharding Architecture



6.3 Leader-Follower Replication



6.4 Caching Patterns

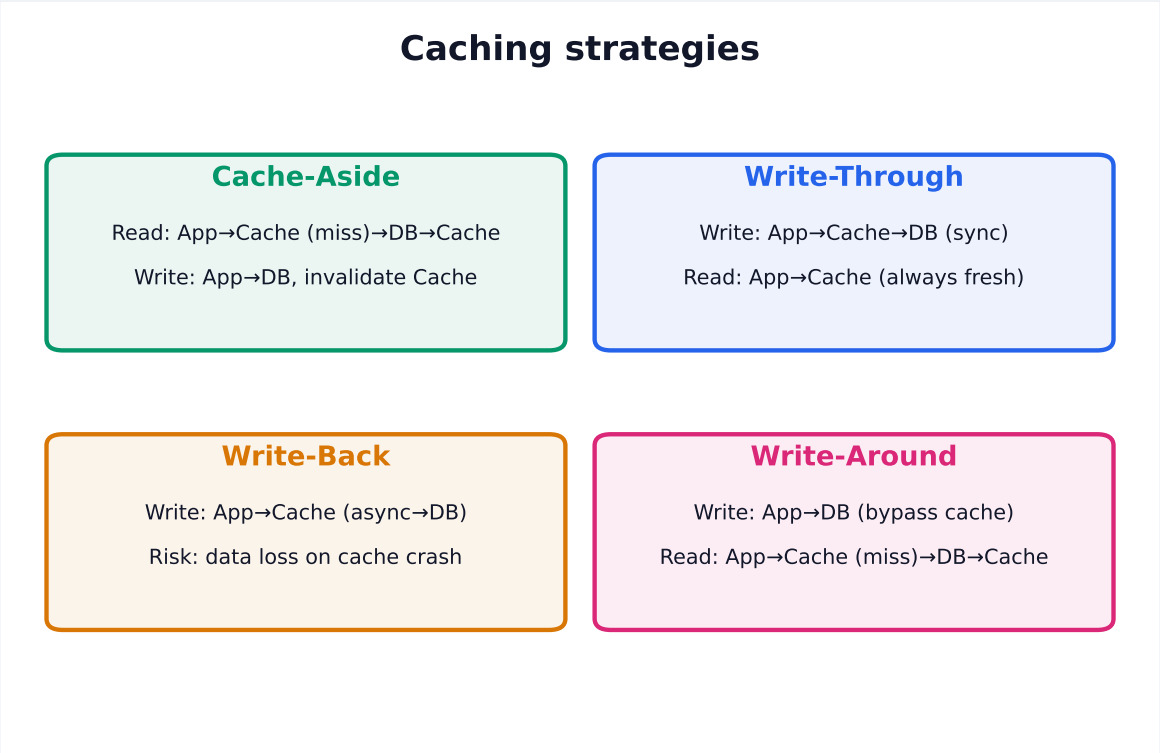
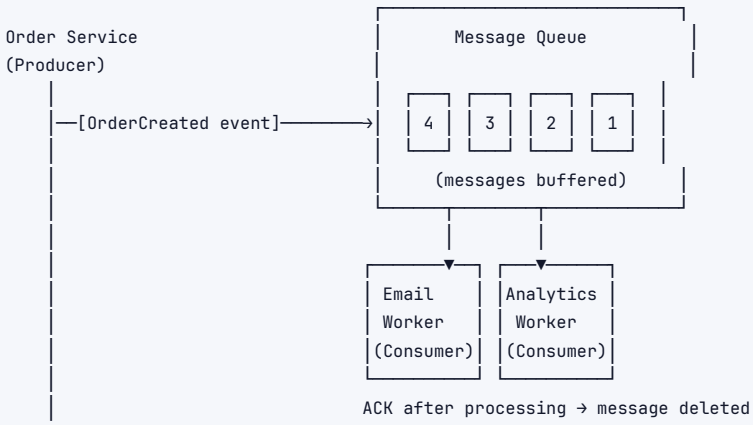


Figure 3. The four write/read cache patterns trade freshness, latency and durability. Cache-aside is the default; write-through favours consistency; write-back favours write latency.

6.5 Message Queue Flow



7 Back-of-Envelope Estimation Cheatsheet

7.1 Latency Numbers Every Programmer Should Know

Operation	Latency		Notes
L1 cache reference	0.5	ns	
Branch mispredict	5	ns	
L2 cache reference	7	ns	14x L1
Mutex lock/unlock	25	ns	
Main memory reference	100	ns	20x L2, 200x L1
Compress 1K bytes with Zippy	3,000	ns	(3 µs)
Send 1K bytes over 1 Gbps network	10,000	ns	(10 µs)

```
Read 4K randomly from SSD           150,000   ns (150 µs)  ~1GB/s SSD
Read 1 MB sequentially from memory  250,000   ns (250 µs)
Round trip within same datacenter   500,000   ns (0.5 ms)
Read 1 MB sequentially from SSD   1,000,000   ns   (1 ms)   4x memory
Disk seek                          10,000,000   ns  (10 ms)
Read 1 MB sequentially from disk   20,000,000   ns  (20 ms)  80x memory
Send packet CA → Netherlands → CA 150,000,000   ns (150 ms)
```

Key takeaways from latency numbers:

- Memory is **200x faster** than random disk
- SSD is **10-30x faster** than spinning disk
- Same-DC call: ~0.5ms; Cross-continent: ~150ms
- **Cache hit (~0.5ms) vs DB query (~1-10ms) = 2-20x difference**

7.2 Powers of 2

```
Power   Exact Value    Approx     Storage name
──────────────────────────────────────────────
10         1,024       1 thousand    1 KB
20     1,048,576       1 million     1 MB
30  1,073,741,824      1 billion     1 GB
32  4,294,967,296      4 billion     (IPv4 space)
40                     1 trillion    1 TB
50                     1 quadrillion 1 PB
```

7.3 QPS / Storage Estimation Reference

Python

```python
Time conversions (memorize):
  1 day    = 86,400 seconds ≈ 10^5 seconds
  1 month  = 2.5M seconds
  1 year   = 32M seconds ≈ 3×10^7

QPS estimation formula:
  QPS = (Users × Actions per User per Day) / 86,400

Storage estimation formula:
  Storage/day = writes/sec × bytes/write × 86,400
  Storage/year = Storage/day × 365
```

7.4 Common System Scale Reference

```
System              DAU       Peak QPS     Storage/day
──────────────────────────────────────────────────────
Twitter             200M      ~350K read   ~25GB (tweets)
Instagram           500M      ~500K read   ~100TB (media)
YouTube             2B        ~1M read     ~3PB (video)
WhatsApp            2B        ~10M msg     ~100TB
Uber (peak)         5M trips  ~100K read   ~10GB
Netflix             200M      ~500K read   ~1PB (content stored)
```

7.5 Bandwidth Math

```python
Network bandwidth:
  1 Gbps  = 125 MB/s
  10 Gbps = 1.25 GB/s

SSD sequential: ~500 MB/s – 3 GB/s
Memory sequential: ~10-50 GB/s
```

8 Real-World Examples

8.1 Design TinyURL

Core features: Shorten URLs, redirect on click, optional custom alias, analytics.

Estimation:

```
Write: 100M URLs/day → ~1,200/sec
Read: 10B redirects/day → ~115,000/sec (100:1 read:write)
Storage: 100M × 500 bytes = 50 GB/day
```

API:

```
POST /shorten  { long_url, custom_alias? } → { short_url }
GET  /{code}   → 302 redirect to long_url
```

Data model:

```
URLs table:
  code        VARCHAR(7)  PK  (Base62: a-z, A-Z, 0-9 → 62^7 = 3.5T combos)
  long_url    TEXT
  user_id     INT
  created_at  TIMESTAMP
  expires_at  TIMESTAMP
```

Code generation approaches:

1. **Hash (MD5/SHA256) + take 7 chars:** Collision risk, need to handle
2. **Counter + Base62 encode:** Simple, predictable (guessable), need distributed counter
3. **Pre-generated pool:** Worker pre-generates codes into DB, pick one on create. No collision.

Architecture:

```
User → API Gateway → App Servers → Cache (Redis: code→URL)
                                      ↓ (miss)
                                PostgreSQL (with read replicas)
```

Redirect flow: GET /{code} → check Redis → hit: return URL → miss: DB query → cache, return URL → 302 redirect.

Interview deep-dives: Custom domains, expiry, analytics (click count, geo), abuse prevention (block malicious URLs).

8.2 Design a News Feed (Instagram/Twitter)

Core features: Post content, follow users, see home feed (posts from followed users, reverse-chron or ranked).

The core problem: Given 500M DAU, user follows 200 people each, each posts 1x/day → 200 posts to potentially insert per user per day → 500M × 200 = 100B feed entries/day.

Two approaches to feed generation:

Fan-out on Write (Push model):

```
User posts → App Server → for each follower → write tweet to follower's feed cache
```

- Reads are O(1) (just read pre-built feed)
- Write amplification: celebrity with 50M followers → 50M writes per tweet
- **Good for:** Most users (low follower count)

Fan-out on Read (Pull model):

```
User requests feed → App Server → get following list → merge N timelines → sort → return
```

- No write amplification
- Read is expensive (must merge N sorted lists)
- **Good for:** Celebrities

Hybrid approach (Twitter's solution):

- For normal users: push model
- For celebrities: pull model (inject celebrity tweets at read time)
- Threshold: ~10K followers → switch to pull

Feed ranking: Not pure reverse-chron anymore. ML models score posts by:

- Engagement prediction (like/comment/share probability)
- Recency
- User relationships (close friends rank higher)
- Diversity (avoid same-author runs)

8.3 Design a Rate Limiter

Token bucket in Redis:

```
GET user:123:tokens  → current tokens
If tokens ⩾ 1:
  DECR user:123:tokens
  Allow request
Else:
  Rate limit response (429)

Background refill job: every second, INCRBY tokens up to max
```

Sliding window in Redis:

```python
key = "ratelimit:user:123"
now = current_timestamp_ms
window = 60000  # 1 minute

ZREMRANGEBYSCORE key 0 (now - window)     # remove old entries
ZADD key now now                           # add current request
count = ZCARD key
EXPIRE key window/1000

If count > limit: reject
```

9 Real-Life Analogies

One growing city — every concept is a road, a shop, or a depot in the same metropolis.

Concept	Real-Life Analogy
Load Balancer	The traffic officer at the city's main junction, waving each car toward the least-busy road so no single lane grinds to a halt
Cache	The corner shop two streets from your house that stocks the twenty items locals buy every day — you skip the forty-minute drive to the distant warehouse (the DB) for a pint of milk
CDN	Neighbourhood mini-marts stocked in every district — nobody drives downtown for everyday goods; the nearest shelf serves you in seconds
Sharding	Splitting the city into postal districts (North, East, South, West), each served by its own depot; a parcel for the North never touches the South depot
Replication	Copies of the city's land records kept in every district town hall — the central hall (leader) stamps every deed first, then branch halls (followers) update their copies so locals can query without crossing town
CAP Theorem	When the bridge linking the East and West districts collapses (partition), the district bank either closes its West branch rather than risk stale balances (consistency) or keeps the branch open on yesterday's ledger (availability) — it cannot do both
Circuit Breaker	The district substation that trips automatically when the grid is overloaded, cutting power to protect the wiring; after a safe interval it resets and reconnects — rather than letting the whole city burn
Message Queue	The city's pneumatic-tube dispatch system: a business drops a sealed canister (message) into the tube; the central sorting office holds it until the right depot (consumer) is ready to collect — the sender doesn't wait at the tube
Rate Limiting	The toll booth on the city highway: each driver gets a book of ten toll tokens per day; once the book is empty the barrier stays down, keeping the road from seizing up
Consistent Hashing	Addresses along the city ring-road: depots are spaced around the loop, and deliveries go to the nearest clockwise depot. Open a new depot and only the handful of deliveries just past it reroute — the rest of the city carries on unchanged
Leader-Follower	The central city records office (leader) is the only place allowed to register a new deed; district town halls (followers) replicate every entry so residents can look up records locally without queuing downtown
Event-Driven Arch	The city's emergency-broadcast loudspeaker: when a water main bursts (event), the broadcast fires once and every crew — plumbers, traffic wardens, road-repair gangs — reacts independently without waiting for a dispatcher to call each one
Microservices	The city's specialist trade streets — the electricians' quarter, the glaziers' row, the plumbers' lane — each guild operates its own yard, hires its own workers, and a fire in one yard doesn't shut down the others
Write-through Cache	The corner-shop owner who updates the shelf tag (cache) and the stock ledger at the depot (DB) in the same stroke — both records stay in sync the moment goods arrive
Write-back Cache	The shop assistant who scribbles each sale on a notepad (cache) and submits a tally to the depot ledger (DB) only at end of day — faster for customers, but a flooded shop loses the day's notes before they're filed
Idempotency	The pedestrian crossing button at a city intersection — pressing it ten times is identical to pressing it once; the crossing changes when it's ready, not once per press
Eventual Consistency	The city's notice-board network: a planning decision is pinned at the central hall first; within hours every district board carries the same notice — briefly, a resident in the outer suburbs hasn't seen it yet, but they will

10 Memory Tricks / Mnemonics

CAP Theorem: "**C**onsistency is **A**lways **P**artial in distributed systems" — you must pick between C and A when P happens.

ACID vs BASE: "**ACID** is for bankers (strict), **BASE** is for beaches (relaxed)."

Caching strategies (CWWW): "**C**ache-aside, **W**rite-through, **W**rite-back, **W**rite-around" — remember by water volume: Cache-aside fills only on demand; Write-through fills both pools simultaneously; Write-back fills fast pool first, slow pool later; Write-around fills slow pool first.

LRU vs LFU: "**R**ecently used = **R**elax (evict old) vs **F**requently used = **F**avor (keep popular)."

Sharding decision rule: "C-A-P-S"

- **C**apacity exceeded? Consider sharding
- **A**ccess patterns known? Choose shard key accordingly
- **P**artition tolerance: now you have it, and all its complexity
- **S**harding is a last resort — exhaust other options first

Load balancing algorithms: "RLWIH" (Round, Least-conn, Weighted, IP-hash, Hash) — "Really Large Websites Implement Hash-routing."

System design interview steps: "RRADE-BD"

- **R**equirements
- **R**ough estimation
- **A**PI design
- **D**ata model
- **E**ntity relationships
- **B**ig-picture architecture
- **D**ive deep + bottlenecks

SQL vs NoSQL rule of thumb: "If you need **J**oins, **T**ransactions, or **S**chema → **SQL** (**JTS = SQL**). If you need **S**cale, **F**lexibility, or **S**pecial access patterns → **NoSQL** (**SFS = NoSQL**)."

Replication sync vs async: "**S**ync = **S**afe (no data loss, slower); **A**sync = **A**gile (faster, risk data loss)."

Message queue choices: "**K**afka for **K**ilo-messages-per-second streaming; **R**abbitMQ for **R**outing complexity; **S**QS for **S**erverless simplicity."

11 Common Interview Questions

11.1 1. Design a URL Shortener (TinyURL / bit.ly)

Approach: Hash vs counter, Base62 encoding, DB schema, redirect (301 vs 302), analytics, expiry, custom aliases.

Key decisions:

- 301 (permanent) redirect → browser caches, reduces future load but no analytics
- 302 (temporary) redirect → analytics works but more server load

Follow-ups: Rate limiting per user, analytics dashboard, abuse detection, multi-DC.

11.2 2. Design Twitter / Social Media Feed

Approach: Fan-out on write vs read, hybrid model for celebrities, feed ranking, post storage, media CDN.

Key decisions: Push vs pull trade-off at what follower threshold; eventual consistency acceptable for feeds.

Follow-ups: Real-time updates (WebSocket/SSE), trending topics, notification system, search.

11.3 3. Design a Chat System (WhatsApp / Slack)

Approach: WebSocket for real-time bidirectional; message storage (Cassandra: write-heavy, time-ordered); delivery receipts; online presence; group chat fan-out.

Key decisions: Server-side fan-out vs client-side; push notifications for offline users (APNs/FCM).

Follow-ups: End-to-end encryption, media sharing, message search, read receipts at scale.

11.4 4. Design YouTube / Video Streaming

Approach: Video upload pipeline (chunked upload → encoding workers → CDN); streaming (HLS/DASH adaptive bitrate); metadata DB; recommendation system.

Key decisions: Encoding at multiple resolutions in parallel; CDN PoPs for low-latency streaming.

Follow-ups: Live streaming, comments, subscription notifications, copyright detection.

11.5 5. Design Uber / Ride-Sharing

Approach: Real-time driver location (WebSocket, update every 4s); geospatial index (geohash/quadtree); matching algorithm; surge pricing; trip state machine.

Key decisions: How to efficiently query "drivers near user" (geohash range query); update frequency vs battery drain.

Follow-ups: ETA calculation, route optimization, driver-side vs rider-side consistency, payment processing.

11.6 6. Design a Rate Limiter

Approach: Token bucket vs sliding window; Redis atomic operations; per-user / per-IP / per-API-key; distributed rate limiting across multiple API servers.

Key decisions: Sliding window log (accurate, more memory) vs sliding window counter (approximate, less memory).

Follow-ups: Rate limit headers (X-RateLimit-Remaining), different limits per tier, bypass for internal services.

11.7 7. Design a Distributed Cache (Redis)

Approach: Cache cluster (sharding via consistent hashing); eviction policies; replication for HA; persistence options (AOF vs RDB); pub/sub for invalidation.

Key decisions: Cache-aside vs write-through; TTL strategy; cache warming on startup.

Follow-ups: Cache stampede prevention, multi-region cache consistency, Redis Sentinel vs Cluster.

11.8 8. Design a Search System (type-ahead / full-text)

Approach: Trie for autocomplete (in-memory or stored); Elasticsearch for full-text; inverted index; ranking (TF-IDF, BM25, ML re-ranking); real-time indexing pipeline.

Key decisions: Trie vs DB prefix query for autocomplete; sharding Elasticsearch by index/shard.

Follow-ups: Personalized results, spell correction, synonyms, content freshness.

12 Senior-Level Discussion Points

12.1 Trade-offs Senior Engineers Must Articulate

Consistency vs Availability:

> "In our checkout flow, we use strong consistency for inventory (can't oversell) but eventual consistency for product recommendations. The choice is driven by business impact of inconsistency."

Sync vs Async:

> "Payment processing is synchronous because the user needs confirmation. Email notification is async because 100ms delay is acceptable and we can't block payment on email infrastructure."

Normalization vs Denormalization:

> "We denormalize the author name into the post object to avoid a JOIN on every feed read. The cost is: on author name change, we must update every post. Since name changes are rare and feed reads are billions/day, this trade-off is worth it."

12.2 Failure Modes

Cascading failures: Service A calls Service B, which is slow → A's thread pool fills waiting → A becomes slow → upstream C fills up → full outage.

- **Solution:** Timeouts + circuit breakers + bulkheads (isolate thread pools per downstream)

Hot spots: User ID 1 = Justin Bieber with 100M followers. All writes fan out to shard holding his followers.

- **Solution:** Detect hot keys, route to dedicated shard/cache, use virtual nodes in consistent hashing

Split-brain: Network partition → both sides elect a new leader → two leaders accepting writes → data divergence.

- **Solution:** Quorum (majority must agree), fencing tokens, STONITH

Clock skew: Distributed system timestamps can't be trusted for ordering.

- **Solution:** Logical clocks (Lamport clocks), vector clocks, Google TrueTime (atomic + GPS clocks with bounded uncertainty)

12.3 Back-Pressure

Problem: Producers emit faster than consumers can process → queue grows unbounded → OOM crash.

Solutions:

1. **Push back to producer:** HTTP 503, return RateLimit headers
2. **Drop oldest messages:** For non-critical streams (metrics)
3. **Prioritize queues:** Critical messages bypass backlogged queues
4. **Scale consumers:** Auto-scale consumer group based on queue depth
5. **Circuit breaker at producer:** Stop producing if consumer lag exceeds threshold

In Kafka: Monitor consumer lag. Alert when lag > threshold. Auto-scale consumer group.

12.4 Write Amplification in Social Systems

The math: 100M users follow 200 people. When a celebrity (10M followers) posts:

- Fan-out on write: 10M DB writes
- At 1 post/hour: 10M × 24 = 240M writes/day from one user

Solutions:

1. Hybrid: fan-out-on-write for normal users, fan-out-on-read for celebrities
2. Lazy fan-out: pre-populate top N followers' caches, rest pull on demand
3. Tiered storage: hot followers in Redis, cold followers in Cassandra

12.5 Observability at Scale

Three pillars:

- **Metrics:** QPS, latency (p50/p95/p99), error rate, saturation → Prometheus + Grafana
- **Logs:** Structured JSON logs → Elasticsearch/Splunk
- **Traces:** Distributed request tracing → Jaeger/Zipkin/AWS X-Ray

SLO vs SLA vs SLI:

- **SLI** (Indicator): Actual measurement (latency p99 = 50ms)
- **SLO** (Objective): Target you set (p99 < 100ms)
- **SLA** (Agreement): Contract with customer (p99 < 200ms, else credit)

Error budget: If SLO = 99.9% availability → 43.8 min/month downtime budget. Spend it on risky deploys.

13 Typical Mistakes Candidates Make

13.1 1. Jumping to solution before requirements

Wrong: "OK so we'll use Kafka and Cassandra and microservices—" **Right:** "Before I design anything, let me make sure I understand the scale and requirements. Can I ask a few questions?"

13.2 2. Over-engineering from the start

Wrong: Proposing 15 microservices, multi-region, event sourcing + CQRS for a URL shortener. **Right:** Start simple (monolith + single DB), then scale only the bottlenecks you identify.

13.3 3. Not doing capacity estimation

Wrong: "We'll just add more servers if needed." **Right:** "At 100M DAU with 10 requests each = 1M RPM = ~17K QPS. We need ~17 servers at 1K QPS each, plus 3x for redundancy and headroom."

13.4 4. Being vague about trade-offs

Wrong: "We'll use caching to make it faster." **Right:** "We'll use Redis with cache-aside pattern. The trade-off is: writes update DB first, so cache may serve stale data for up to 5 minutes (our TTL). This is acceptable for user profiles but not for inventory."

13.5 5. Forgetting failure modes

Wrong: Designing the happy path only. **Right:** After high-level design: "Now let me stress-test this. What happens if the cache fails? If the primary DB goes down? If the message queue gets backed up?"

13.6 6. Wrong database choice

Wrong: ”Let's use MongoDB because it's NoSQL and more scalable.” (Cargo-culting) **Right:** ”Given we have complex queries with joins, strong consistency requirements for transactions, and our team knows SQL — PostgreSQL is the right choice. If we hit read scale limits, we'll add read replicas. Sharding would come much later.”

13.7 7. Not handling the "celebrity problem"

Wrong: Designing fan-out-on-write without acknowledging write amplification for popular users. **Right:** ”For the 1% of users with >10K followers, we'll use fan-out-on-read and inject their posts at query time.”

13.8 8. Ignoring network partitions

Wrong: Assuming the network is always reliable. **Right:** ”Given network partitions will happen, we have to choose CP or AP. For this payment system, we choose CP — we'd rather return an error than process a payment on stale data.”

13.9 9. Not communicating clearly

Wrong: Silent thinking for 3 minutes. **Right:** Think out loud continuously: ”I'm considering two approaches here: X and Y. X has this trade-off, Y has this trade-off. Given our consistency requirement, I'll go with Y.”

13.10 10. Forgetting about data at rest

Wrong: Only modeling the API and compute. **Right:** ”Let me also model the data. What's the schema? How do we handle indexing? What's our backup strategy? How do we handle schema migrations?”

14 How This Connects To Other Topics

14.1 Distributed Systems

System design *is* distributed systems applied. CAP theorem, consensus (Raft/Paxos), vector clocks, CRDTs — all foundational. A distributed system must handle: network partitions, clock skew, node failures, Byzantine failures (in blockchain-style systems).

14.2 Databases

- SQL/NoSQL choice dictates your sharding strategy, consistency model, and query patterns
- B-trees (SQL indexes), LSM-trees (Cassandra, RocksDB) — storage engine choice affects write vs read performance
- MVCC (Multi-Version Concurrency Control) — how PostgreSQL achieves isolation without locking reads
- Write-ahead logging (WAL) — enables crash recovery and replication

14.3 Computer Networks

- TCP vs UDP choice matters for real-time systems (games, video: UDP; reliability: TCP)
- HTTP/1.1 vs HTTP/2 vs HTTP/3 — multiplexing, header compression, QUIC protocol
- TLS handshake cost → terminate at load balancer, not at each microservice
- DNS TTL → how long before a failed-over IP propagates

14.4 Cloud / Infrastructure

- AWS: EC2 (compute), RDS/Aurora (managed SQL), DynamoDB (managed NoSQL), S3 (object storage), ElastiCache (managed Redis/Memcached), SQS/SNS/Kafka-MSK (queuing), CloudFront (CDN), ALB/NLB (load balancers), Lambda (serverless)
- Auto-scaling groups: scale out on CPU/QPS metric → handles load spikes
- Multi-AZ deployment: replication across availability zones for HA
- Multi-region: active-active or active-passive for DR and global latency

14.5 Performance Engineering

- Profiling before optimizing: measure P99 latency, not average
- Amdahl's Law: speedup limited by non-parallelizable fraction
- Little's Law: $N = \lambda \times W$ (concurrency = arrival_rate × service_time) → crucial for capacity planning

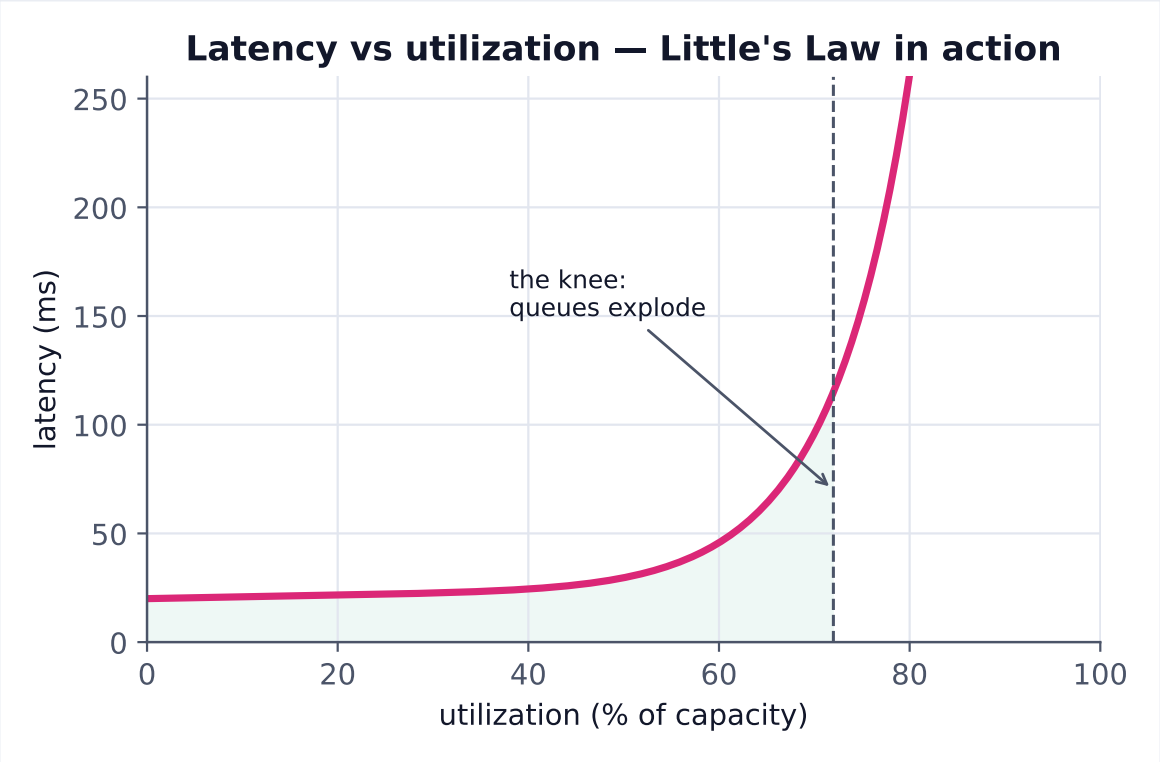


Figure 4. Latency stays flat until utilization approaches capacity, then rises sharply as queues build. Plan for headroom — running hot (>70-80%) is a tail-latency trap.

- Working set size vs available memory → cache hit rate prediction

14.6 Security

- API Gateway as the security perimeter (auth, WAF, DDoS protection)
- Zero-trust networking: even internal services must authenticate
- Data encryption at rest (AES-256) and in transit (TLS 1.3)
- Secrets management: never hardcode credentials → Vault, AWS Secrets Manager

15 FAANG Interview Tips

15.1 Before the Interview

1. **Practice on a whiteboard** (or Excalidraw/miro) — interviews happen on whiteboards, not IDE
2. **Time-box your sections** — 5 min requirements, 5 min estimation, 10 min high-level, 15 min deep-dive
3. **Know 5-8 systems cold:** URL shortener, news feed, chat, search, rate limiter, notification system, distributed cache, ride-sharing
4. **Read the system design primer:** github.com/donnemartin/system-design-primer
5. **Read actual engineering blogs:** Uber Engineering, Airbnb Tech, Netflix TechBlog, AWS Architecture Blog

15.2 During the Interview

1. **Clarify before designing** — Never assume scale or requirements
2. **Think out loud always** — "I'm considering X vs Y because..."
3. **Draw first, explain second** — Sketch the architecture, then explain each component
4. **Drive the interview** — Have opinions, don't wait to be asked
5. **Quantify everything** — Don't say "faster," say "reduces latency from 50ms to 2ms"
6. **Name the trade-offs explicitly** — "The downside of this approach is..."
7. **Acknowledge what you'd do differently at 10x scale**
8. **Ask for feedback** — "Is there a specific area you'd like me to go deeper on?"

15.3 Red flags interviewers look for:

- No requirements gathering
- Jumping to buzzwords (Kubernetes, microservices) without justification
- Can't explain why they chose SQL vs NoSQL
- No mention of failure cases
- Can't estimate scale
- Vague on data model
- Passive, not leading the conversation

15.4 Green flags that signal senior-level thinking:

- Proactively identifies bottlenecks before being asked
- Knows specific numbers (latency, capacity)
- Discusses operational concerns (monitoring, deployment, rollback)
- References real systems and what they learned from them
- Explicitly states assumptions and asks if they're correct
- Brings up failure scenarios and how to handle them

16 Revision Cheat Sheet

16.1 10-Minute Summary

System design = Requirements + Scale Estimation + API + Data Model + Architecture + Trade-offs

1. **Always clarify:** DAU, QPS, consistency, latency, availability requirements
2. **Estimate first:** QPS, storage, bandwidth — round aggressively
3. **Standard architecture:** Client → CDN → LB → App (stateless) → Cache → DB → Queue → Workers

4. **Scaling order:** Vertical → Read replicas → Caching → Sharding → Microservices
5. **DB choice:** SQL for ACID + joins; NoSQL for horizontal scale + flexible schema
6. **Caching strategies:** Cache-aside (most common), Write-through (consistency), Write-back (performance)
7. **CAP theorem:** Must pick CP (consistency, for banking) or AP (availability, for social)
8. **Message queues:** Decouple producers/consumers; Kafka for streaming, SQS for simple
9. **Microservices:** Independent deployability + DB-per-service = complexity; start with monolith
10. **Senior signals:** Failure modes, back-pressure, hot spots, observability, trade-off articulation

16.2 Key Points

Topic	One-liner
Horizontal scaling	Stateless app tier + external session store
Load balancing	L4 for TCP, L7 for HTTP routing + A/B
Cache-aside	App manages cache; misses hit DB
Write-through	Both cache and DB updated on write
LRU eviction	Evict least recently accessed
CDN	Cache static content geographically near users
Sharding	Partition data across nodes; consistent hashing minimizes resharding
Replication	Copy data across nodes; sync=safe, async=fast
CAP	Pick 2 of 3; P is mandatory; choose C or A
Eventual consistency	Replicas converge given no new writes
Idempotency	Safe to retry; use idempotency keys
Circuit breaker	Fail fast when downstream is unhealthy
Message queue	Decouple, buffer, fan-out; at-least-once + idempotent consumer
API Gateway	Auth, rate limit, routing, SSL in one place
Rate limiting	Token bucket (burst-friendly) or sliding window (precise)

16.3 Cheat-Sheet Table: The Most Important Decisions

Decision	Rule of Thumb
SQL vs NoSQL	SQL default; NoSQL when horizontal scale or special access pattern needed
Cache strategy	Cache-aside default; write-through for high consistency; write-back for high write throughput
Sync vs Async	Sync for user-facing responses; async for background work, fan-out, decoupling
Fan-out on write vs read	Write for low-follower-count users; read for celebrities; hybrid for both
Sharding key choice	High cardinality, uniform distribution, matches access pattern (avoid cross-shard queries)
Replication sync vs async	Sync for financial data; async for social/analytics
CAP choice	CP for consistency-critical (banking, inventory); AP for availability-critical (social, search)
Monolith vs microservices	Monolith to start; microservices when teams/services need independent scaling/deployment

Decision	Rule of Thumb
REST vs gRPC	REST for public APIs; gRPC for internal microservice communication
Push vs Pull notifications	Push (FCM/APNs) for mobile; SSE/WebSocket for web real-time

16.4 Most Important Concepts (Rank-ordered by interview frequency)

1. **Horizontal scaling + stateless app servers** — foundation of everything
2. **Caching (cache-aside, LRU, TTL, invalidation)** — most common optimization lever
3. **Database choice (SQL vs NoSQL + why)** — asked in every design
4. **Load balancing (L4 vs L7, algorithms)** — part of every architecture
5. **CAP theorem + consistency models** — senior expectation
6. **Message queues + async processing** — Kafka/SQS in every large system
7. **Sharding + consistent hashing** — needed for any at-scale DB discussion
8. **CDN** — must mention early, serves static content, reduces origin load
9. **Rate limiting** — common standalone design question + mentioned in every API design
10. **Replication + leader-follower** — must know for DB deep-dives
11. **Microservices trade-offs** — monolith vs microservices discussion in every senior loop
12. **API Gateway** — entry point for security, routing, rate limiting
13. **Back-of-envelope estimation** — first thing to do in every design
14. **Circuit breaker + failure handling** — senior green flag
15. **Idempotency** — payment systems, retry safety

Study tip: Practice narrating each design out loud for 45 minutes. Silence is your enemy in a system design interview. Build the habit of communicating continuously.